Domain-Specific Fine-Tuning for Reliability Engineering: From Small-Seed Self-Instruct to Reinforcement Learning

Alexandre Dalban
CentraleSupélec, Université Paris-Saclay
alexandre.dalban@student-cs.fr

Elora Drouilhet
CentraleSupélec, Université Paris-Saclay
elora.drouilhet@student-cs.fr

Zhiguo Zeng
CentraleSupélec, Université Paris-Saclay
Laboratoire Génie Industriel (LGI)
zhiguo.zeng@centralesupelec.fr

Abstract

Large language models show strong general reasoning but struggle with domain-specific technical tasks such as reliability engineering, where answers require precise numerical computation and specialized knowledge. We present a complete pipeline for adapting a general-purpose LLM to the reliability engineering domain, starting from only 56 seed questions extracted from textbooks. We introduce an adapted self-instruct methodology that replaces same-model answer consistency with cross-model verification, combined with paraphrase-based augmentation inspired by MetaMath, to scale the dataset to 866 high-quality question-reasoning-answer triplets. We identify the self-instruct ceiling effect, wherein LLM generators tend to produce questions systematically easier than real domain problems. Supervised fine-tuning of Qwen3-8B with QLoRA yields a statistically significant $+18.6\%$ *accuracy improvement (*$p = 0.002$*) under 10-fold cross-validation, placing our 8B-parameter model between frontier models o3-mini and Gemini 3.1 Pro on reliability engineering tasks. We further apply Group Relative Policy Optimization (GRPO) with verifiable rewards on 266 "mixed-difficulty" questions, achieving an additional* $+7.5$ *percentage points over the base model on in-distribution questions. Our ablation study reveals a paradoxical role for SFT: it degrades standalone performance on hard questions (*-20.4*pp on holdout) yet is essential as GRPO initialization (*$+7.1$*pp vs. GRPO from scratch). Crucially, GRPO preserves base-model generalization on a 54-question independent holdout (53.7%), fully recovering the severe catastrophic forgetting caused by SFT alone (*-20.4*pp), demonstrating that RL from verifiable rewards can improve domain performance without the out-of-distribution cost of supervised fine-tuning.*

1. Introduction

Reliability engineering is concerned with the analysis and prediction of failure in complex systems, from nuclear power plants and aircraft to electrical grids and autonomous vehicles. Practitioners routinely employ sophisticated mathematical tools including Weibull and exponential lifetime distributions, Markov chain modeling, Bayesian inference, fault tree analysis, and system-level reliability computation for series-parallel and k-out-of-n architectures. Mastering these techniques requires years of specialized training, and even experienced engineers benefit from computational assistance when solving multi-step quantitative problems.

Large language models (LLMs) have demonstrated impressive general reasoning capabilities across a wide range of tasks [11]. However, when applied to domain-specific technical problems such as reliability engineering, their performance degrades substantially. Prior work evaluated state-of-the-art models on a set of hand-written reliability engineering problems and found that even frontier models fail to produce correct numerical answers in the majority of cases, often generating plausible-sounding but mathematically incorrect solutions. This performance gap motivates the development of domain-adapted models that can reliably assist with reliability engineering computations.

A central challenge in building such models is the scarcity of training data. Unlike well-studied domains such

*Code and data: https://github.com/ADnocap/Reliability-Domain-Specific-LLM

*Code and data: https://github.com/helloelora/grpo-reliability-engineering

†Experiments (exp) described in appendix A

as general mathematics [7, 25] or medicine [19], reliability engineering lacks large-scale question-answer datasets or established benchmarks. Manually creating such datasets is prohibitively expensive, as each problem requires expert knowledge to both formulate and solve. This creates a bootstrapping problem: we need data to train a model, but generating quality data requires the very expertise the model is meant to provide.

We address this challenge with a data generation and fine-tuning pipeline that bootstraps 56 textbook-extracted seed questions into 866 verified training examples, achieving $+18.6\%$ accuracy ($p = 0.002$) on Qwen3-8B. Our contributions are:

- A cross-model answer verification method for synthetic data quality control, replacing same-model consistency checks that we show are trivially satisfied in technical domains (Section 3.2).
- The identification and empirical characterization of the self-instruct ceiling effect (Section 3.2), and evidence that paraphrase augmentation overcomes it more effectively than difficulty scaling (Section 3.3).
- A reproducible SFT pipeline achieving $+18.6\%$ on reliability engineering with a fine-tuned 8B model competitive with frontier models an order of magnitude larger (Section 4).
- Integration of GRPO with verifiable rewards and DAPO-style dynamic sampling [26] to improve domain reasoning by $+7.5$pp beyond the base model, with an ablation showing that SFT warm-start is essential for RL effectiveness despite degrading standalone performance (Section 5).

2. Related Work

Domain-specific LLM adaptation. Fine-tuning LLMs for specialized domains has shown promising results in medicine [19, 23], law [2], and code [16]. A common finding is that domain adaptation requires high-quality, domain-specific training data. For niche technical fields like reliability engineering, no such datasets or benchmarks exist, motivating our data generation pipeline.

Self-Instruct and synthetic data. Self-Instruct [22] bootstraps instruction-following data from a small seed set, with extensions including Alpaca [20], WizardLM [24], and Self-Refine [15]. For verifiable tasks, answer-consistency filtering [22] has been proposed, but we show in Section 3.2 that same-model consistency is trivially satisfied in technical domains. SearchInstruct [1] identifies that self-instruct "depends solely on internal knowledge," motivating external grounding. We address correctness through *cross-model* verification (Section 3.2.1).

Data augmentation for reasoning. MetaMath [25] showed that rephrasing questions from multiple perspectives improves reasoning more than increasing difficulty, with PersonaMath [10] extending this via persona-driven variants. Shumailov *et al*. [18] formalized "model collapse" risks when training on generated data. Our paraphrase strategy (Section 3.3) is inspired by MetaMath with cross-model verification to prevent quality degradation.

Reinforcement learning for reasoning. RLHF [12] and DPO [14] align models with preferences but rely on noisy preference data. For verifiable tasks, GRPO [17] generates multiple responses and computes group-relative advantages without a critic model. DeepSeek-R1 [3] scaled this to elicit sophisticated reasoning. DAPO [26] introduced dynamic sampling—discarding prompts with zero reward variance—which is critical for small datasets. We apply GRPO with DAPO-style sampling to reliability engineering (Section 5), showing that SFT warm-start is essential for RL effectiveness.

Parameter-efficient fine-tuning. We use 4-bit QLoRA [4, 8] with NEFTune [9] regularization, applied to all attention and MLP layers.

3. Data Pipeline

Our pipeline transforms 56 textbook-extracted seed questions into 866 verified training examples through four stages: seed extraction, synthetic generation with cross-model verification, cleaning, and paraphrase augmentation. Figure 1 illustrates the complete flow.

3.1. Seed Extraction

We digitized 32 chapters from two reliability engineering textbooks—*Applied Reliability* (3rd ed.) by Tobias and Trindade [21] and *Reliability Engineering* (3rd ed.) by Elsayed [5]—using Mistral OCR. From this corpus, we extracted **56 seed QRA triplets** via regex scanning for exercise blocks, LLM-assisted validation for self-containment, and structured extraction into (q, r, a) format.

3.2. Cross-Model Verification

Unlike Self-Instruct [22], we use a *stronger* generator (Claude Opus 4) to create training data for the smaller Qwen3-8B target. Each generation prompt includes 3 randomly sampled seeds and requests 2 new problems with full chain-of-thought reasoning (temperature 0.8). Quality gates filter out short questions, placeholder answers, and near-duplicates. This yielded 158 synthetic QRA triplets.

3.2.1. Verification Pipeline

To ensure correctness, we replace same-model consistency with cross-model verification: (1) Claude Opus 4 gener-

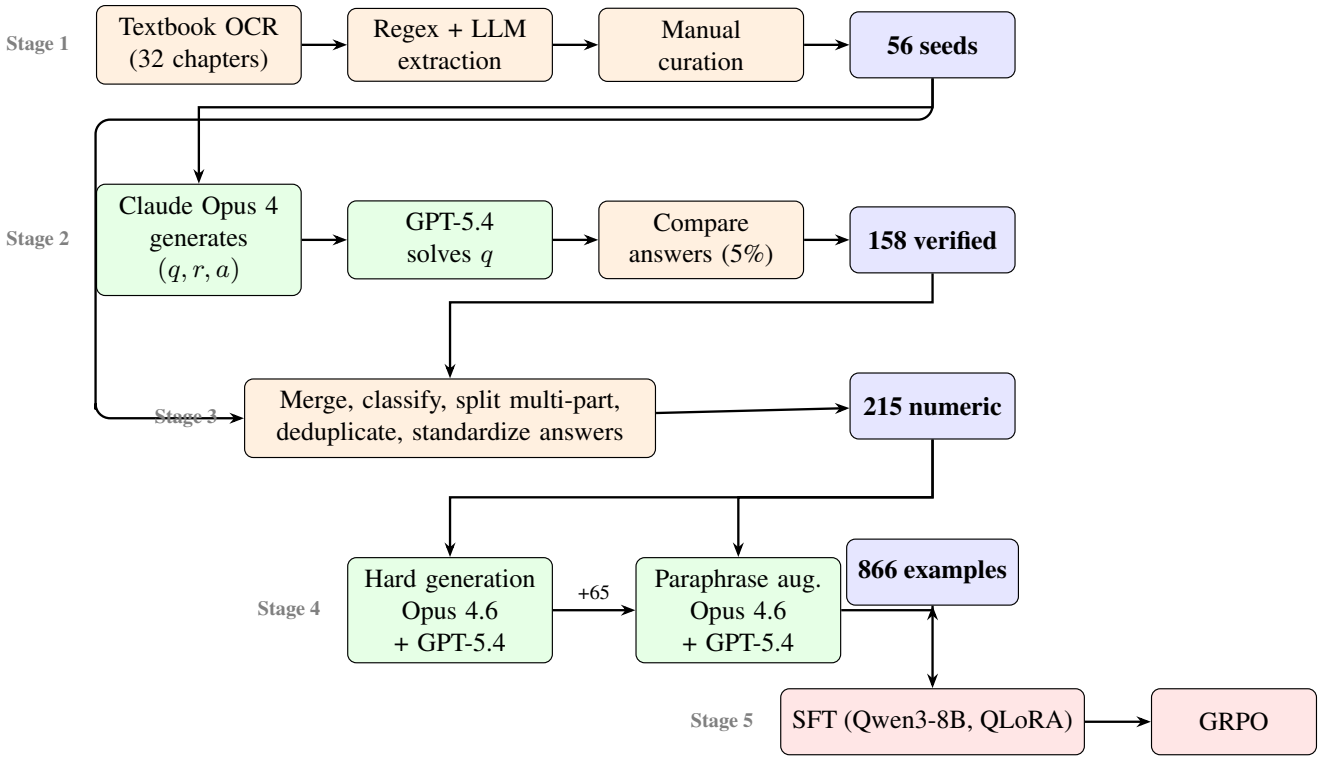

Figure 1. Data pipeline overview. Blue: dataset milestones; orange: processing; green: LLM-based generation/verification; red: training.

ates (q, r, a) from seeds; (2) GPT-5.4 independently solves q without seeing the answer or reasoning; (3) numerical answers are compared within 5% tolerance, with LLM-based semantic comparison as fallback. Only matching items are retained. Because the verifier is a different model with different failure modes, agreement provides a stronger correctness signal than same-model consistency.

3.2.2. Self-Instruct Ceiling Effect

We observe that baseline models achieve 92–97% accuracy on self-instruct-generated questions while scoring only 68–76% on textbook problems, meaning same-model answer consistency [22] is trivially satisfied. We term this the *self-instruct ceiling effect*: generators create questions within their own capability envelope, avoiding edge cases they cannot solve. This motivates our paraphrase strategy.

3.3. Cleaning and Augmentation

The combined seed and synthetic data underwent LLM-assisted cleaning using Claude Opus 4.6: type classification, splitting of 107 multi-part questions into self-contained entries, answer standardization (stripping units, normalizing LaTeX), and deduplication by prefix matching. This yielded **215 numeric questions**. We then augment via two strategies:

- **Hard generation:** 65 new difficult questions (Claude Opus 4.6, verified by GPT-5.4).
- **Paraphrase augmentation** (inspired by MetaMath [25]): Claude Opus 4.6 generates surface-level rephrases (changing wording, scenario framing) while strictly preserving all numerical values and formulas. Each rephrase is verified by GPT-5.4 for numerical and mathematical equivalence.

Table 1. Dataset versions. All entries are verified numeric QRA triplets.

Version	Size	Source
v1 (cleaned)	215	98 textbook + 158 verified
v2 (+ hard)	280	v1 + 65 hard generated
v3 (+ para.)	501	v2 + 221 paraphrased
v4 (+ para.)	**866**	**v2 + 586 paraphrased**

The contrast is striking: adding 65 hard questions yields only $+2.0\%$, while 586 paraphrases yield $+18.6\%$ ($p = 0.002$), confirming that surface diversity outperforms difficulty scaling.

3.4. Dataset Summary

4. Supervised Fine-Tuning

4.1. Model Selection

We evaluated two open-source instruction-tuned models as base candidates: Llama 3.1 8B [6] and Qwen3-8B [13], both in 4-bit quantized form via Unsloth. Qwen3-8B achieves a substantially higher zero-shot baseline on our reliability engineering questions (70.7% vs. 37.5%), likely due to its stronger mathematical pre-training. Moreover, fine-tuning Llama 3.1 8B caused a regression on numeric questions (-1.4pp) while improving only format-dependent categories (formula $+28.6$pp, text $+15.8$pp), suggesting catastrophic forgetting of mathematical capabilities. We therefore selected Qwen3-8B as our base model for all subsequent experiments.

By analyzing the model's thinking traces, we observed that the base model uses these blocks as *scratch work*: it explores solution avenues, backtracks when an approach seems wrong, and iterates toward the answer, much like a human solving a problem for the first time. Our SFT training data, by contrast, consists of polished step-by-step solutions that proceed directly to the correct answer without false starts. Training on these clean reasoning chains with thinking enabled would effectively require the model to "already know the answer" and go straight to a proof, conflicting with the exploratory nature of the thinking trace. We found that this mismatch caused the model to overfit, producing empty thinking tags followed by shallow pattern-matched answers. Disabling thinking during both training and inference resolved this: the model generates direct chain-of-thought responses consistent with the training format, while preserving its internal reasoning capabilities for use during GRPO (Section 5).

4.2. Training Setup

4.2.1. LoRA Configuration

We use 4-bit QLoRA [4] via the Unsloth framework, which provides optimized CUDA kernels for approximately 40% faster training. LoRA adapters [8] are applied to all attention and MLP projection layers. Key hyperparameters are rank $r = 16$, scaling factor $\alpha = 32$, and dropout 0.05.

4.2.2. Training Hyperparameters

Table 2 summarizes the training configuration. NEFTune [9] adds uniform noise to embedding vectors during training, serving as a regularizer against memorization on small datasets. We found $\alpha = 5$ to be optimal: $\alpha = 7$ degraded performance by -1.3pp, and $\alpha = 10$ showed only marginal gains. Early stopping with patience 2 (monitoring validation loss on a 10% held-out split) consistently converged to epoch 4 regardless of the maximum epoch setting (6 or 8), indicating a stable training dynamic.

Table 2. SFT training configuration (best setup).

Parameter	Value
Learning rate	2×10^{-4} (cosine)
Optimizer	AdamW 8-bit
Batch size	2 (grad. accum. 4, eff. 8)
Max epochs	8 (early stop $\rightarrow$ epoch 4)
Warmup steps	5
Weight decay	0.01
NEFTune α	5
Precision	BF16
Max sequence length	2048 tokens

4.2.3. Data Format

Training examples follow a three-turn chat format: a *system* message establishing the reliability engineering expert persona, a *user* message containing the question, and an *assistant* message containing the full chain-of-thought reasoning followed by the final answer. The tokenizer's `apply_chat_template()` handles model-specific formatting.

4.3. Evaluation Protocol

4.3.1. K-Fold Cross-Validation

Given the small dataset sizes (215–866 examples), we use k-fold cross-validation rather than a single train/test split to obtain reliable performance estimates. Initial experiments used 5-fold CV (172 train / 43 test per fold), but 5 folds limit the minimum achievable p-value to 0.0625 under the Wilcoxon test, precluding statistical significance at $p < 0.05$. We therefore moved to 10-fold CV for experiments on 866 samples ($\sim$780 train / $\sim$87 test per fold), which achieves $p = 0.002$ when all folds show improvement.

For each fold, a fresh base model is loaded and fine-tuned from scratch to avoid information leakage between folds. All evaluation uses greedy decoding (`do_sample=False`) with `max_new_tokens=4096`, ensuring deterministic and reproducible results. Early experiments with stochastic decoding inflated improvements (e.g., $+7.0\%$ stochastic vs. $+2.3\%$ greedy).

4.3.2. Answer Comparison

We extract the final answer from model responses using pattern matching ("Final Answer:", "The answer is:", "Therefore,") with fallback to the last non-empty line. Extracted answers are then compared against ground truth through a multi-level pipeline:

1. **Exact match:** Normalized strings (lowercase, strip whitespace, remove LaTeX wrappers such as `\boxed{}`, convert `\frac{a}{b}` to `a/b`).
2. **Numerical match:** Extract all numbers via regex; accept if all values match within 5% relative tolerance.

Table 3. SFT results across dataset sizes (best config per size). All use Qwen3-8B, greedy decoding, LR=2×10^{-4}, NEFTune=5.

N	Folds	Ep.	Base	FT	Δ	p
215	5	4	70.7	73.0	$+2.3$	0.50
280	5	4	71.0	73.0	$+2.0$	0.69
501	5	4	59.1	65.3	$+6.2$	0.063
866	**10**	**8→4**	**63.6**	**82.2**	$\mathbf{+18.6}$	**0.002**

3. **Fraction-to-decimal:** Convert fractions (e.g., 14/33 ↔ 0.4242).
4. **Percentage conversion:** Normalize percentages (e.g., 25.5% ↔ 0.255).

An answer is considered correct if any of the above criteria is satisfied.

4.3.3. Statistical Testing

We use the Wilcoxon signed-rank test to compare per-fold baseline and fine-tuned accuracies. This non-parametric paired test is appropriate for small paired samples where normality cannot be assumed. We use $p < 0.05$ as the significance threshold. We additionally track *question flips*—questions that change correctness between baseline and fine-tuned models—as a measure of improvement robustness.

4.4. SFT Results

Table 3 presents the main SFT results across dataset sizes. The improvement scales non-linearly with dataset size: from $+2.3\%$ (not significant) on 215 original questions to $+18.6\%$ ($p = 0.002$) on 866 augmented questions. All 10 folds showed improvement, with per-fold deltas ranging from $+12.6\%$ to $+26.4\%$. This suggests a threshold effect around 500 examples, below which LoRA fine-tuning struggles to overcome the regularization of frozen base weights.

The question flip ratio is particularly informative: at 215 samples, the model trades nearly as many correct answers as it gains (1.3:1, 23 W→R vs. 18 R→W), while at 866 samples, gains overwhelm regressions (4.6:1, 206 W→R vs. 45 R→W), confirming that paraphrase augmentation produces robust improvements. Note that baseline accuracy is lower on larger datasets (63.6% on 866 vs. 70.7% on 215) because augmented test sets include paraphrased questions the base model has not seen.

4.5. Ablation Studies

Table 4 summarizes key ablation results on the 215-question dataset (5-fold CV, greedy decoding).

LR=2×10^{-4} consistently outperforms 1×10^{-4}, consistent with the recommendation that LoRA adapters benefit from learning rates approximately 10× higher than full fine-tuning. Early stopping consistently selects epoch 4 across all dataset sizes. Both DPO (-1.7%) and early GRPO ($+0.5\%$) underperform pure SFT on 215 questions; the early GRPO used TRL with LLM-judge rewards, motivating our improved pipeline with verifiable rewards on a larger training set (Section 5).

Table 4. SFT ablations on 215 questions (5-fold CV, greedy).

Variation	Δ	p
Best (LR=2×10^{-4}, NEF=5, 4ep)	$\mathbf{+2.3}$	0.50
Lower LR (1×10^{-4})	-0.5	1.0
Higher NEFTune ($\alpha=7$)	-1.3	0.38
Lower LoRA rank ($r=8$, $\alpha=16$)	$+1.4$	0.88
3 epochs (vs. 4)	$+0.9$	0.88
5 epochs (vs. 4)	$+1.9$	0.50
DPO (instead of SFT)	-1.7	0.63
SFT + GRPO (early)	$+0.5$	0.88

5. Reinforcement Learning with GRPO

5.1. Motivation

While SFT teaches the model domain-specific formatting and reasoning patterns, it is fundamentally limited to imitating the training data. On questions the base model already solves correctly, SFT reinforces existing capabilities; on questions where the model fails, SFT provides the correct answer but does not teach the model *how to arrive at it* from its own reasoning process. This limitation is reflected in our results: the SFT model trained on the 600 "easy" questions (those the base model solves 4/4 times) shows no improvement and even slight regression on the remaining 266 harder questions (Section 5.7).

Group Relative Policy Optimization (GRPO) [17] offers a complementary approach: rather than imitating expert solutions, the model generates multiple candidate answers and learns to prefer reasoning chains that lead to correct results. For reliability engineering, where answers are verifiable numerical values, this provides a natural reward signal without requiring a learned reward model.

5.2. GRPO Algorithm

GRPO generates G completions $\{o_1, \ldots, o_G\}$ for each prompt q, scores each with a reward function $r(o_i, q)$, and computes *group-relative* advantages:

$$\hat{A}_i = \frac{r(o_i, q) - \bar{r}}{\sigma_r + \epsilon}, \tag{1}$$

where $\bar{r}$ and σ_r are the mean and standard deviation of rewards within the group. The policy is updated to maximize:

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{G}\sum_{i=1}^{G} \hat{A}_i \cdot \frac{\log \pi_\theta(o_i \mid q)}{|o_i|}, \tag{2}$$

where $|o_i|$ is the response length in tokens, normalizing the loss to prevent bias toward shorter completions (Dr. GRPO [3]).

Unlike PPO, GRPO requires no critic model: the group mean serves as the baseline. Unlike DPO, it does not require pre-computed preference pairs but generates them online. This makes GRPO particularly suited to domains with verifiable rewards and limited data.

5.3. Dynamic Sampling (DAPO)

A key challenge with GRPO on small datasets is that many prompts produce zero reward variance: either all G generations are correct (no learning signal for improvement) or all are wrong (no positive signal). Following DAPO (Dynamic sampling Policy Optimization) [26], we implement *dynamic sampling*: when all G generations produce the same reward, the prompt is discarded and a new one is sampled. This ensures every training step provides a meaningful gradient signal.

We track *useful steps* (prompts with reward variance) separately from *wasted steps* (prompts discarded due to zero variance). The waste rate serves as a diagnostic: a rising waste rate indicates the model is converging (more prompts become too easy or too hard), while an initially high waste rate suggests the training distribution is poorly calibrated to the model's current capability.

5.4. Reward Design

Our reward function uses the same automated numerical comparison as the SFT evaluation pipeline (Section 4.3). Given a model response o and ground-truth target y, we extract the last `\boxed{}` value $\hat{y}$ and compute a graded reward based on relative error $\epsilon = |\hat{y} - y|/(|y| + 10^{-9})$:

$$r(o, q) = \begin{cases} 1.0 & \text{if } \epsilon \leq 0.1\% \\ 0.8 & \text{if } \epsilon \leq 1\% \\ 0.4 & \text{if } \epsilon \leq 5\% \\ 0.01 & \text{if } \epsilon > 5\% \\ 0.0 & \text{if no } \backslash\texttt{boxed}\{\} \text{ found} \\ -0.3 & \text{if } > 2 \text{ boxed values} \end{cases} \tag{3}$$

The graded structure provides a smoother reward landscape than binary correct/incorrect: partially correct answers ($\epsilon \leq 5\%$) receive positive reward, encouraging the model to refine its computation rather than abandon an approach entirely. The penalty for multiple `\boxed{}` values discourages hedging.

5.5. Training Data Split

The 866-question dataset (v4) is split based on base-model screening: each question is evaluated 4 times with the base Qwen3-8B model, and questions are categorized by success rate:

Table 5. Accuracy on 266 mixed questions. Δ and p (McNemar) are vs. SFT fold_4.

Model	Acc.	Trunc.	Δ	p
Qwen3-8B base	50.4%	4	—	—
SFT fold_4	50.8%	8	ref.	—
GRPO from scratch, step 80	50.8%	1	+0.0	1.00
GRPO from SFT, step 200	52.3%	6	+1.5	—
GRPO from SFT, step 100	54.1%	9	+3.3	—
GRPO from SFT, step 80	**57.9%**	10	**+7.1**	0.056

- **600 questions (4/4 correct):** Used for SFT training. These are "easy" for the base model; SFT reinforces correct patterns.
- **266 questions (1/4 to 3/4 correct):** Used for GRPO training. These have natural reward variance—some generations succeed, others fail—making them ideal for group-relative optimization.

Questions where the base model scores 0/4 are excluded from GRPO as they provide no positive reward signal. This split ensures zero data leakage between SFT and GRPO stages and targets GRPO training at the model's learning frontier—questions that are neither trivially easy nor impossibly hard.

5.6. Training Configuration

We use a custom training loop (no TRL dependency) with $G = 4$ generations per prompt, 200 useful steps, LR=1×10^{-5} with cosine decay, gradient accumulation of 4, and $\beta = 0$ (no KL penalty). The pipeline loads Qwen3-8B, applies the SFT fold_4 LoRA ($r = 16$, $\alpha = 32$, ~43.6M trainable parameters), and runs GRPO with dynamic sampling on the 266 mixed questions. Full configuration details are in Appendix A.

5.7. GRPO Results

5.7.1. In-Distribution Evaluation (266 Mixed Questions)

Table 5 presents accuracy on the 266 GRPO training questions across all model configurations. All evaluations use deterministic seeding (seed $= 3407 + q_i$) with temperature 1.0 and top-p 0.95, matching the GRPO generation settings.

The best configuration (GRPO from SFT at step 80) achieves **57.9%** accuracy, +7.1pp over the SFT fold_4 starting point (McNemar $p = 0.056$, approaching significance with 54 wins vs. 35 losses). Performance degrades with continued training: -3.8pp from step 80 to step 100 and -5.6pp to step 200, indicating overfitting after approximately 80 useful steps on this 266-question dataset.

5.7.2. Out-of-Distribution and Ablation Results

Table 6 presents two key analyses: (a) generalization on 54 independent holdout questions never seen during SFT

Table 6. (a) Holdout accuracy (54 unseen questions) and (b) SFT warm-start ablation (step 80, 266q). p: McNemar test.

(a) Independent holdout (54 questions)				
Model	Cor.	Acc.	Δ	p
Qwen3-8B base	29	53.7%	—	—
SFT fold_4	18	33.3%	-20.4	0.019
GRPO from SFT, step 80	29	53.7%	$+0.0$	0.804
GRPO from scratch, step 200	30	55.6%	$+1.9$	1.00
(b) SFT warm-start ablation (266q, step 80)				
Init	Acc.	Δ	Waste	p
Fresh LoRA	50.8%	ref.	29.3%	—
SFT fold_4	**57.9%**	$\mathbf{+7.1}$	23.1%	0.104

or GRPO training, and (b) the effect of SFT warm-start on GRPO effectiveness.

Holdout generalization. GRPO from SFT at step 80 achieves 53.7%, matching the base model exactly ($p = 0.804$). While the in-distribution gain does not transfer, crucially it does not regress either. The SFT fold_4 alone drops to 33.3% (-20.4pp, $p = 0.019$), a *statistically significant* degradation. GRPO fully recovers this catastrophic forgetting ($p = 0.023$ vs. SFT, 11 wins vs. 2 losses). GRPO from scratch at step 200 achieves 55.6% ($+1.9$pp), though this amounts to a single additional question.

Caveat on holdout size. With only 54 questions, each question represents $\sim$1.85pp of accuracy, limiting statistical power. We deliberately chose not to augment the holdout via paraphrasing: while this would increase sample size, it would introduce repeated mathematical content and risk inflating accuracy through surface-level memorization. A larger holdout composed of *distinct* problems would strengthen these findings.

SFT warm-start ablation. The SFT warm-start is essential: GRPO from scratch barely exceeds the base model ($+0.4$pp, likely within noise), while GRPO from the SFT checkpoint gains $+7.5$pp. The SFT initialization also produces a lower waste rate (23.1% vs. 29.3%), indicating more diverse reward signals and more efficient dynamic sampling. This is paradoxical: the SFT alone degrades holdout performance by -20.4pp, yet it provides a critical initialization for GRPO. We hypothesize that SFT teaches domain-specific *format and vocabulary* (reliability formulas, structured computation) without teaching correct *application* to hard problems. GRPO then leverages this structural foundation to optimize for numerical correctness. Without SFT, the LoRA starts at zero and must simultaneously learn both format and reasoning—too many degrees of freedom for 266 questions.

Table 7. Frontier and fine-tuned model accuracy on reliability engineering benchmarks.

Model	v4 (500q)	Holdout (54q)
Claude Sonnet 4.6	**92.4%**	**81.1%**
o3-mini	86.4%	77.1%
Gemini 3.1 Pro	76.4%	82.6%†
Qwen3-8B (base)	63.6%*	53.7%
Qwen3-8B (SFT)	82.2%*	33.3%
Qwen3-8B (SFT+GRPO)	57.9%**	53.7%

*10-fold CV. **On 266 mixed-difficulty questions. †23/54 extracted.

We also trained with LR=5×10^{-6} and 2×10^{-5}; training dynamics (waste rates 20.9–23.1%) were similar across all three learning rates. Full results are in Appendix A.

6. Experiments and Analysis

6.1. Comparison with Frontier Models

To contextualize our fine-tuned 8B model, we evaluate three frontier models on a 500-question sample from v4 and on the 54-question independent holdout (Table 7).

Our SFT model (82.2% under 10-fold CV) places between o3-mini and Gemini 3.1 Pro despite being an order of magnitude smaller. On the holdout, SFT regresses sharply to 33.3% (-20.4pp, $p = 0.019$), while GRPO fully recovers to 53.7%, matching the base model (Section 5.7).

6.2. Analysis

Dataset size scaling. SFT improvement scales nonlinearly with dataset size (Table 3): marginal below 300 examples, a threshold effect around 500, and strong gains at 866 ($+18.6\%$, $p = 0.002$). The question flip ratio grows from 1.3:1 (215 samples) to 4.6:1 (866 samples), indicating that larger augmented datasets produce more robust improvements with fewer catastrophic regressions.

Epoch convergence. Early stopping (patience 2) consistently selects epoch 4 across all dataset sizes and configurations, confirming a stable convergence point.

GRPO overfitting. GRPO accuracy degrades with continued training past step 80 (-3.8pp at step 100, -5.6pp at step 200), suggesting 266 questions are insufficient for sustained improvement. The truncation rate increases with training (4 for base, 10 for GRPO step 80), indicating GRPO encourages longer reasoning chains.

7. Conclusion

We presented a complete pipeline for adapting a general-purpose LLM to reliability engineering, a technical domain with no existing training datasets or benchmarks. Starting from 56 textbook-extracted seed questions, our adapted self-instruct methodology with cross-model answer verification and paraphrase augmentation scales the dataset to 866 verified examples. Supervised fine-tuning of Qwen3-8B yields a statistically significant $+18.6\%$ accuracy improvement ($p = 0.002$) under 10-fold cross-validation, placing our 8B-parameter model between frontier models o3-mini and Gemini 3.1 Pro on domain-specific tasks.

Our analysis reveals two key findings for the synthetic data generation community. First, the *self-instruct ceiling effect*: LLM generators create questions within their own capability envelope, making same-model answer consistency a trivially satisfied check rather than a meaningful quality filter. Second, paraphrase augmentation—creating surface-level rephrases while preserving mathematical content—is far more effective than generating harder synthetic questions, consistent with MetaMath [25] and suggesting that diversity of expression matters more than difficulty in small-data regimes.

GRPO with verifiable rewards achieves an additional $+7.5$pp on the 266 mixed-difficulty training questions, demonstrating that reinforcement learning can improve reasoning beyond SFT on verifiable tasks. Our ablation study reveals a paradoxical role for the SFT stage: while SFT alone causes catastrophic forgetting on unseen questions (-20.4pp on holdout), it is essential as initialization for GRPO ($+7.1$pp vs. GRPO from scratch). We hypothesize that SFT provides structural alignment (domain vocabulary, format, formula usage) that GRPO requires as a foundation for reasoning optimization.

Importantly, GRPO preserves base-model generalization: on the 54-question independent holdout, the GRPO model matches the base model (53.7%), fully recovering the severe catastrophic forgetting caused by SFT alone (-20.4pp). This suggests that GRPO provides in-distribution gains without the out-of-distribution cost typically associated with fine-tuning on small datasets.

Limitations. Our evaluation is limited to numeric questions with automated answer comparison; formula derivation and conceptual questions require human evaluation. The 54-question holdout is small (each question ≈ 1.85pp), limiting statistical power for out-of-distribution conclusions. The self-instruct ceiling effect means our synthetic data may not cover the hardest problems in reliability engineering.

Ethical and societal impact. Reliability engineering directly concerns the safety of critical infrastructure—nuclear power plants, aircraft, electrical grids—where incorrect calculations can have catastrophic consequences. While our work aims to make domain expertise more accessible, deploying an LLM as a computational assistant in safety-critical contexts carries inherent risks. A model that produces plausible but incorrect answers could lead practitioners to accept erroneous failure predictions or underestimate system risk. We therefore emphasize that our model is a research prototype, not a certified engineering tool, and should not replace expert judgment or verified analytical software in operational settings. Any practical deployment would require rigorous validation against established standards and human oversight. On the positive side, by lowering the barrier to reliability computations, such tools could democratize access to safety analysis capabilities—particularly for smaller organizations or developing regions that lack specialized reliability engineers—ultimately contributing to safer engineered systems.

Future work. Promising directions include: (1) scaling the GRPO training set via paraphrase augmentation of the 266 mixed questions, following the successful SFT scaling pattern; (2) introducing a KL penalty ($\beta > 0$) to reduce overfitting and preserve base model generalization; (3) early stopping based on holdout validation rather than training loss; (4) grounding data generation in textbook source material (following SearchInstruct [1]); and (5) extending to non-numeric question types and other engineering domains.

References

[1] Iman Barati, Mostafa Amiri, and Heshaam Faili. SearchInstruct: Enhancing domain adaptation via retrieval-based instruction dataset creation. *arXiv preprint arXiv:2509.10708*, 2025. 2, 8

[2] Jiaxi Cui, Zongjian Li, Yang Yan, Bohua Chen, and Li Yuan. ChatLaw: Open-source legal large language model with integrated external knowledge bases. *arXiv preprint arXiv:2306.16092*, 2023. 2

[3] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. 2, 6

[4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. *NeurIPS*, 2023. 2, 4

[5] Elsayed A. Elsayed. *Reliability Engineering*. John Wiley & Sons, 3rd edition, 2012. 2

[6] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. 4

[7] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob

Steinhardt. Measuring mathematical problem solving with the MATH dataset. *NeurIPS*, 2021. 2

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022. 2, 4

[9] Neel Jain, Ping-yeh Chiang, Yuxin Wen, John Kirchenbauer, Hong-Min Chu, Gowthami Somepalli, Brian R. Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Aniruddha Saha, et al. NEFTune: Noisy embeddings improve instruction finetuning. *arXiv preprint arXiv:2310.05914*, 2023. 2, 4

[10] Jing Luo, Run Luo, Longze Chen, Liang Zhu, Chang Ao, Jiaming Li, et al. PersonaMath: Enhancing math reasoning through persona-driven data augmentation. *arXiv preprint arXiv:2410.01504*, 2024. 2

[11] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 1

[12] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022. 2

[13] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. 4

[14] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *NeurIPS*, 2023. 2

[15] Leonardo Ranaldi and André Freitas. Self-refine instruction-tuning for aligning reasoning in language models. *arXiv preprint arXiv:2405.00402*, 2024. 2

[16] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023. 2

[17] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeek-Math: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. 2, 5

[18] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The curse of recursion: Training on generated data makes models forget. *Nature*, 631:755–759, 2024. 2

[19] Karan Singhal, Tao Tu, Juraj Gottweis, Rory Sayres, Ellery Wulczyn, Le Hou, Kevin Clark, Stephen Pfohl, Heather Cole-Lewis, Darlene Neal, et al. Towards expert-level medical question answering with large language models. *arXiv preprint arXiv:2305.09617*, 2023. 2

[20] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford Alpaca: An instruction-following LLaMA model. *GitHub repository*, 2023. 2

[21] Paul A. Tobias and David C. Trindade. *Applied Reliability*. Chapman & Hall/CRC, 3rd edition, 2012. 2

[22] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *ACL*, 2023. 2, 3

[23] Chaoyi Wu et al. PMC-LLaMA: Toward building open-source language models for medicine. *Journal of the American Medical Informatics Association*, 2024. 2

[24] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. WizardLM: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023. 2

[25] Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. MetaMath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*, 2024. 2, 3, 8

[26] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, et al. DAPO: An open-source LLM reinforcement learning system via dynamic sampling. *arXiv preprint arXiv:2503.14476*, 2025. 2, 6

A. Complete Experiment Log

Table 8 lists all 26 experiments with full configurations and results. All experiments use 4-bit QLoRA via Unsloth on NVIDIA A100-SXM4-40GB GPUs. Evaluation uses greedy decoding (`do_sample=False`) from Exp. 7 onwards; earlier experiments used stochastic sampling (marked with †). Bold rows indicate best results per dataset size. ES = early stopping (patience=2); the notation "8(ES→4)" means max 8 epochs with early stopping selecting epoch 4.

Table 9 lists all GRPO experiments. All use the custom DAPO-style training loop on 266 mixed-difficulty questions (unless noted), with Qwen3-8B 4-bit and LoRA $r = 16/\alpha = 32$. Evaluation uses deterministic seeding (seed $= 3407 + q_i$), temperature 1.0, top-p 0.95.

Notes: Exp. 1 uses Llama 3.1 8B; all others use Qwen3-8B. Exp. 5 uses LoRA $r = 8/\alpha = 16$; all others use $r = 16/\alpha = 32$. Exp. 10 timed out after 14 hours. Exp. 13 crashed due to Triton kernel incompatibility. Exp. 14 completed only 4 of 5 folds (missing adapter for fold 4). Accuracy values are percentages; Δ is the absolute difference (FT − Baseline). W→R and R→W are question flip counts aggregated across all folds.

B. Key Prompts

Below are selected prompts used in our data pipeline. Minor formatting adjustments have been made for readability.

B.1. Synthetic Question Generation

Used to generate new reliability engineering problems from seed examples (Section 3.2).

> *You are an expert in reliability engineering, creating practice problems for students.*
>
> *I will show you {num_examples} example problems. Your task is to:*
>
> 1. *Understand the style, difficulty, and topics of these examples*
> 2. *Create {num_to_generate} new, original problems that are similar in style and difficulty*
> 3. *Make questions that are educational and challenging*
> 4. *Provide complete solutions with step-by-step reasoning*
>
> *Important: Create different questions (don't just change numbers in the examples). Use similar concepts but new scenarios. Include all necessary data in the question (self-contained). Show clear reasoning steps. Provide specific numerical answers when appropriate.*
>
> *[3 randomly sampled seed examples shown]*
>
> *For each problem, provide:*
> *QUESTION: [Complete problem statement]*
> *REASONING: [Step-by-step solution]*
> *ANSWER: [Final answer(s)]*

B.2. Paraphrase Augmentation

Used to rephrase existing questions while preserving mathematical content (Section 3.3). The prompt includes two manually curated few-shot examples (omitted for brevity).

> *You are a technical writer specializing in reliability engineering. Your task is to rephrase a question and its solution reasoning while preserving their exact mathematical meaning.*
>
> *Strict rules:*
>
> 1. *All numerical values, failure rates, probabilities, time periods, and parameters must remain exactly the same*
> 2. *All formulas, equations, and mathematical relationships must be preserved*
> 3. *The correct answer must be identical*
> 4. *Change the wording, sentence structure, and phrasing to create a distinct version*
> 5. *Do not add information, assumptions, or conditions not present in the original*
>
> *What to change: Synonyms for non-technical words (e.g., "compute" → "determine"), sentence structure, scenario framing (e.g., "plant" → "factory"), transition words.*
>
> *What must not change: Any number, rate, probability, or parameter value. Variable names in formulas. The mathematical operations and formulas used. The final answer.*
>
> *[2 few-shot examples shown]*
>
> *Respond with a JSON object:* `{"question": "...", "reasoning": "..."}`

B.3. Paraphrase Verification

Used by GPT-5.4 to verify that a rephrase preserves mathematical meaning (Section 3.3).

> *You are verifying whether a rephrased reliability engineering question preserves the exact mathematical meaning of the original. Compare the original and rephrased versions. Check all of the following:*
>
> 1. *Numerical values: Are all numbers, rates, probabilities exactly the same?*
> 2. *Mathematical meaning: Do both questions ask for the exact same quantity?*
> 3. *Constraints: Are all conditions and given information preserved?*
> 4. *Solution: Would the exact same calculation steps produce the correct answer?*
> 5. *Reasoning: Are all formulas and intermediate values identical?*

Table 8. Complete experiment log. All models are Qwen3-8B (4-bit) except Exp. 1 (Llama 3.1 8B). LoRA config is r/α with dropout 0.05 unless noted. † = stochastic eval (inflated). ‡ = thinking enabled (mismatch).

Exp	Dataset	N	Folds	Ep.	LR	NEF	Method	Baseline	FT	Δ	W→R	R→W	p
Round 1: Model Selection													
1	cleaned (all)	256	5	3	2e-4	–	SFT†	37.5	39.8	+2.4	35	29	0.25
2	cleaned (num)	215	5	4	5e-5	–	SFT†‡	67.9	67.0	−0.9	25	27	0.88
Round 2: Hyperparameter Search (stochastic eval)													
3	cleaned (num)	215	5	3	2e-4	5	SFT†	67.9	74.9	+7.0†	31	16	0.13
4	cleaned (num)	215	5	3	2e-4	10	SFT†	70.7	72.1	+1.4	21	18	1.0
5	cleaned (num)	215	5	3	2e-4	5	SFT†	67.9	69.3	+1.4	24	21	0.88
6	cleaned (num)	215	5	3	1e-4	5	SFT†	68.8	68.4	−0.5	20	21	1.0
Round 3: Deterministic Evaluation (greedy decoding)													
7	v2 (num)	280	5	3	2e-4	5	SFT	64.0	65.2	+1.3	21	21	0.88
8	v2 (num)	280	5	5	2e-4	5	SFT	64.3	65.7	+1.4	30	19	0.88
9	cleaned (num)	215	5	5	2e-4	5	SFT†	70.7	72.6	+1.9	21	17	0.50
11	cleaned (num)	215	5	3	2e-4	5	SFT	70.7	71.6	+0.9	22	20	0.88
12	**cleaned (num)**	**215**	**5**	**4**	**2e-4**	**5**	**SFT**	**70.7**	**73.0**	**+2.3**	**23**	**18**	**0.50**
16	**v2 (num)**	**280**	**5**	**4**	**2e-4**	**5**	**SFT**	**71.0**	**73.0**	**+2.0**	**19**	**15**	**0.69**
17	cleaned (num)	215	5	4	2e-4	7	SFT	71.0	69.8	−1.3	19	18	0.38
Round 4: Alternative Methods													
10	v2 (num)	280	5	DPO	5e-6	–	DPO	64.3	–	–	–	–	–
14	cleaned (num)	215	5	DPO	5e-6	–	DPO	69.8	68.0	−1.7	4	7	0.63
13	cleaned (num)	215	5	SFT+GRPO	2e-4	5	GRPO	70.7	–	–	–	–	–
15	cleaned (num)	215	5	SFT+GRPO	2e-4	5	GRPO	70.7	71.2	+0.5	18	17	0.88
Round 5: Paraphrase Augmentation (5-fold CV)													
18	**v3 (num)**	**501**	**5**	**4**	**2e-4**	**5**	**SFT**	**59.1**	**65.3**	**+6.2**	**74**	**43**	**0.063**
19	v3 (num)	501	5	8(ES→4)	1.5e-4	5	SFT	59.1	64.5	+5.4	83	56	0.063
20	v3 (num)	501	5	6	1e-4	5	SFT	59.1	64.9	+5.8	78	49	0.063
Round 6: Scaled Augmentation (10-fold CV)													
23	v4 (num)	866	10	3	2e-4	5	SFT	63.6	75.1	+11.4	171	72	**0.002**
21	v4 (num)	866	10	4	2e-4	5	SFT	63.6	79.3	+15.7	194	58	**0.002**
22	v4 (num)	866	10	6(ES→4)	2e-4	5	SFT	63.6	81.8	+18.1	208	51	**0.002**
24	**v4 (num)**	**866**	**10**	**8(ES→4)**	**2e-4**	**5**	**SFT**	**63.6**	**82.2**	**+18.6**	**206**	**45**	**0.002**
Round 7: 600-Sample Subset Validation (10-fold CV)													
25	**v4 sub (num)**	**600**	**10**	**5**	**2e-4**	**5**	**SFT**	**68.0**	**79.7**	**+11.7**	**108**	**38**	**0.002**
26	v4 sub (num)	600	10	5(ES)	2e-4	5	SFT	68.0	78.8	+10.8	100	35	**0.002**

Table 9. GRPO experiment log. All models are Qwen3-8B (4-bit) with 200 useful steps, $G = 4$ generations, grad_accum=4. Accuracy is on the 266 mixed questions unless noted.

Name	Init	LR	Waste%	Attempts	Step 80	Step 100	Step 200	Holdout (54q)	Notes
GRPO baseline	SFT fold_4	1e-5	23.1	260	**57.9%**	54.1%	52.3%	53.7% (step 80)	Best config
GRPO aggressive LR	SFT fold_4	2e-5	20.9	253	Evaluation pending			—	2× LR
GRPO conservative LR	SFT fold_4	5e-6	21.3	254	Evaluation pending			—	0.5× LR
GRPO from scratch	Fresh LoRA	1e-5	29.3	283	50.8%	Pending	Pending	55.6% (step 200)	No SFT init

Respond with a JSON object:
`{"values_preserved": bool, "meaning_preserved": bool, "constraints_preserved": bool, "solution_equivalent": bool, "reasoning_equivalent": bool, "overall_valid": bool, "issues": "..."}`

B.4. GRPO System Prompt

Used during GRPO training and evaluation (Section 5). The `/no_think` prefix disables Qwen3's internal thinking mode for consistency with SFT format.

/no_think
You are a Reliability Engineering Expert. Solve the user's problem step-by-step with rigorous mathematical reasoning.

Rules for your final answer:

- *Write ONE single* `\boxed{}` *at the very end of your response — your final answer only.*
- *Do NOT use* `\boxed{}` *for intermediate steps or calculations.*

Always put your single final numerical answer inside `\boxed{}`.

B.5. SFT System Prompt

Used as the system message during both training and evaluation (Section 4.2).

> *You are an expert in reliability engineering, probability theory, and system analysis. Provide clear, accurate answers with step-by-step reasoning. At the end, clearly state your final answer after 'Final Answer:'.*

C. Project Organization

C.1. Task Distribution

This project was carried out by two students under the supervision of Prof. Zhiguo Zeng. Table 10 summarizes the main tasks and their distribution.

Task	Alexandre	Elora
Literature review	✓	✓
Seed dataset curation	✓	
Synthetic data generation	✓	
Paraphrase augmentation	✓	
SFT training pipeline	✓	
GRPO implementation	✓	✓
Evaluation & baselines	✓	✓
HPC cluster management	✓	
Paper writing (draft)	✓	
Paper writing (revision)	✓	✓
Figures & tables		✓

Table 10. Task distribution between team members.

C.2. Project Timeline

The project spanned approximately six months, from late October 2025 to April 2026. Figure 2 presents the timeline as a Gantt chart.

C.3. Time Estimates

Table 11 provides approximate time spent on each major phase of the project.

Phase	Approx. hours
Literature review & problem formulation	30
Data pipeline (seed + synthetic + augment.)	60
SFT training & hyperparameter search	50
GRPO implementation & experiments	40
Evaluation & baseline comparison	25
Paper writing & revision	45
Total (combined)	**250**

Table 11. Approximate time spent on each project phase (combined for both team members).

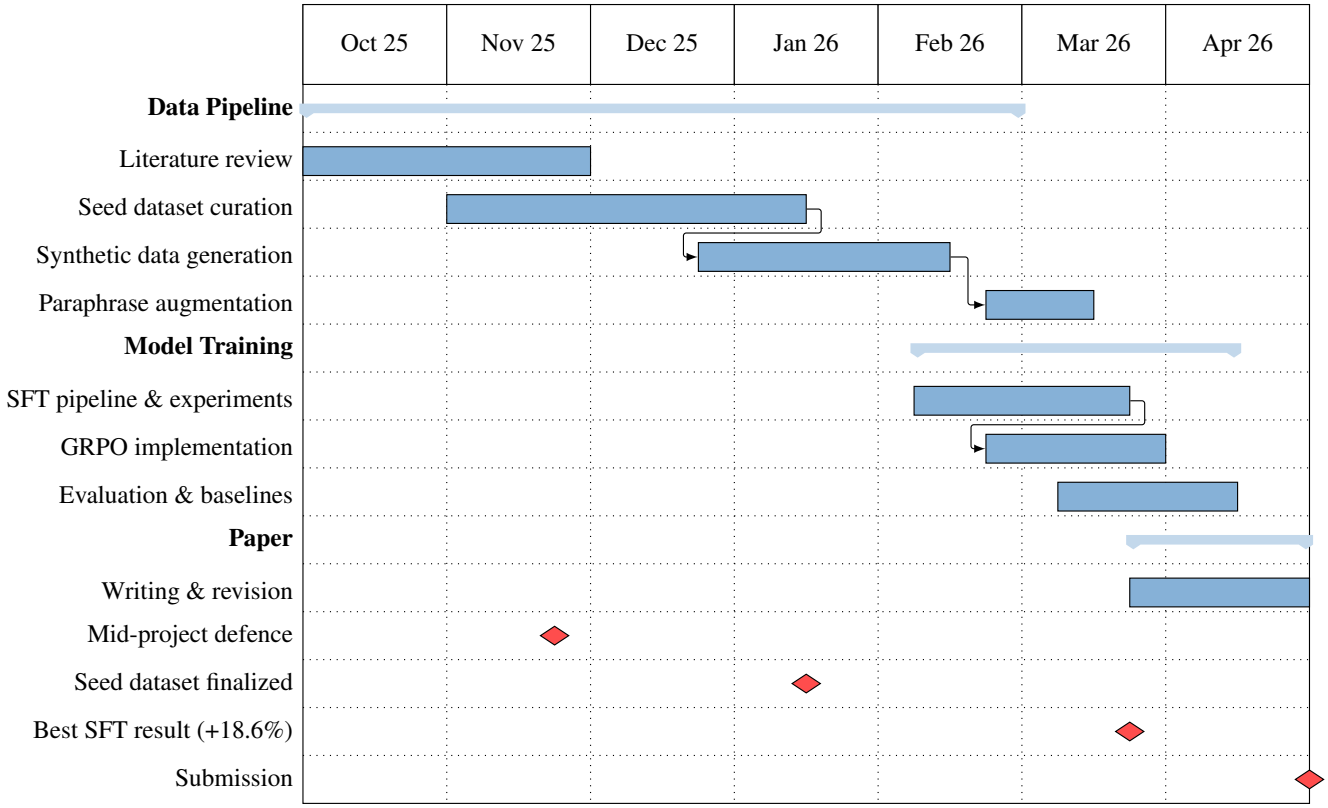

Figure 2. Project timeline (October 2025 – April 2026).